

嵌入 Looker 並使用對話式數據分析

來源：<https://codelabs.developers.google.com/codelabs/looker-embedded-conversational-analytics?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將瞭解如何整合支援對話式數據分析的即時通訊功能，以及內嵌的 Looker 資訊主頁。您將使用 Looker 嵌入參考資料，建構自己的範例應用程式。

目錄

1. [1. 事前準備](#)
2. [2. 設定對話式數據分析](#)
3. [3. 設定 Looker 嵌入參考資料](#)
4. [4. 試用範例](#)
5. [5. 建構即時通訊元件](#)
6. [6. 將資訊主頁篩選器傳送至對話式數據分析](#)
7. [7. 透過即時通訊控制資訊主頁篩選器](#)
8. [8. 結論與重點](#)

步驟 1 · 約 0 分鐘

1. 事前準備

本程式碼研究室將引導您整合嵌入式 Looker 資訊主頁和對話式數據分析功能，打造以自然語言為基礎的統一資料體驗。如要發揮最大價值，您應熟悉 Looker 嵌入、對話式數據分析、JavaScript 和 React。

課程內容

完成本程式碼研究室後，您將學到：

- 如何在本地設定 Looker 嵌入參考應用程式
- 如何使用 Looker 元件庫建構聊天 React 元件
- 如何將內嵌資訊主頁的篩選器做為背景資訊傳送至對話式數據分析
- 如何啟用對話式數據分析功能，透過對話控制內嵌資訊主頁的篩選器

軟硬體需求

如要完成這個程式碼研究室，您必須符合以下條件：

- 已安裝 [ASC 人口統計資料 Looker 模塊](#)，並啟用 [SSO 嵌入](#) 的 Looker 執行個體
 - API 和開發人員對 Looker 執行個體的存取權
 - 已安裝 Node v18、yarn、Git 和 gcloud 的本機環境
 - 為使用者帳戶設定下列 IAM 角色的雲端專案：
 - `roles/bigquery.dataViewer` BigQuery 資料檢視者
 - `roles/bigquery.user` BigQuery 使用者
 - `roles/looker.instanceUser` Looker 執行個體使用者
-

步驟 2 · 約 5 分鐘

2. 設定對話式數據分析

我們來設定對話式數據分析資料代理，嵌入式資訊主頁的對話功能會使用這個代理，以自然語言回答問題。

使用 gcloud 進行驗證

1. 在本機環境中使用使用者帳戶進行驗證：

〇〇〇〇

```
gcloud auth login
```

2. 在 gcloud 中設定應用程式預設憑證 (ADC) 和報帳專案：

〇〇〇〇

```
gcloud auth application-default login  
gcloud auth application-default set-quota-project YOUR_PROJECT_ID
```

啟用對話式數據分析 API

3. 啟用雲端專案 API。請將 `YOUR_PROJECT_ID` 替換為您的 Google Cloud 雲端專案 ID：

〇〇〇〇

```
gcloud services enable geminidataanalytics.googleapis.com bigquery.googleapis.com  
cloudaicompanion.googleapis.com --project=YOUR_PROJECT_ID
```

建立資料代理

4. 開啟 [Google Colab](#)。
5. 在 Google Colab 中，從 [Looker 嵌入參考存放區](#) 載入這個筆記本。
6. 執行筆記本中的所有步驟。您需要雲端專案 ID 和 Looker 執行個體的 URI (結尾須加上斜線)，例如 `"https://my.looker.app/"`。筆記本結尾應會顯示成功結果。

您現在可以使用 [對話式數據分析資料代理程式](#)，接受即時通訊訊息、查詢嵌入式 Looker 資訊主頁中的 Looker 探索，並傳回結果和圖表。

步驟 3 · 約 10 分鐘

3. 設定 Looker 嵌入參考資料

我們將在本機環境中設定 Looker 嵌入參考應用程式，方便您試用將對話式數據分析即時通訊整合至嵌入式 Looker 資訊主頁的範例。

複製存放區

1. 將 [GitHub 存放區](#) 複製到本機環境。請參閱下列指令範例：

```
git clone "https://github.com/looker-open-source/LookerEmbedReference.git"
```

2. 前往您複製的 `LookerEmbedReference` 目錄：

```
cd LookerEmbedReference
```

設定及執行本機前端伺服器

3. 在 `Frontend` 目錄中安裝依附元件

```
cd Frontend  
yarn install
```

4. 在 `Frontend` 目錄根層級中，使用憑證設定 `.env` 檔案。`YOUR_LOOKER_INSTANCE_URI` 應為 Looker 執行個體的 URI，且不得有尾端斜線。檔案內容應如下所示：

```
PBL_CLIENT_PORT=3001  
API_HOST=http://localhost:3000  
LOOKER_HOST=YOUR_LOOKER_INSTANCE_URI  
LOOKER_API_HOST=YOUR_LOOKER_INSTANCE_URI:19999  
LOOKERSDK_EMBED_HOST=YOUR_LOOKER_INSTANCE_URI
```

您可以使用下列範例指令建立檔案：

```
cat > .env <<'EOF'  
PBL_CLIENT_PORT=3001  
API_HOST=http://localhost:3000  
LOOKER_HOST=YOUR_LOOKER_INSTANCE_URI  
LOOKER_API_HOST=YOUR_LOOKER_INSTANCE_URI:19999  
LOOKERSDK_EMBED_HOST=YOUR_LOOKER_INSTANCE_URI  
EOF
```

5. 執行前端開發伺服器：

```
yarn run dev
```

在本機設定及執行後端伺服器

6. 開啟新的殼層、終端機、主控台或分頁。前往 `Backend-Node` 目錄並安裝依附元件。請務必保留先前執行前端伺服器的殼層/終端機。

```
cd ../Backend-Node
yarn install
```

7. 在 `Backend-Node` 目錄根目錄中設定 `.env` 檔案，並加入憑證：

- `YOUR_LOOKER_CLIENT_ID` 是您的 Looker 用戶端 ID。
- `YOUR_LOOKER_CLIENT_SECRET` 是您的 Looker 用戶端密鑰。
- `YOUR_LOOKER_EMBED_SECRET` 是您的[嵌入密鑰](#)。
- `YOUR_PROJECT_ID` 是您的 Cloud 專案 ID。
- `YOUR_LOOKER_INSTANCE_URI_WITH_TRAILING_SLASH` 是 Looker 執行個體 URI，結尾有斜線。
- `YOUR_LOOKER_INSTANCE_URI` 是 Looker 執行個體 URI，**結尾沒有斜線**。

檔案內容應如下所示：

```
PBL_BACKEND_PORT=3000
LOOKERSDK_API_VERSION=4.0
LOOKERSDK_BASE_URL=YOUR_LOOKER_INSTANCE_URI_WITH_TRAILING_SLASH
LOOKERSDK_CLIENT_ID=YOUR_LOOKER_CLIENT_ID
LOOKERSDK_CLIENT_SECRET=YOUR_LOOKER_CLIENT_SECRET

LOOKERSDK_EMBED_HOST=YOUR_LOOKER_INSTANCE_URI
LOOKERSDK_EMBED_SECRET=YOUR_LOOKER_EMBED_SECRET

CLOUD_AGENT_ID=looker_embed_reference_data_agent
CLOUD_PROJECT_ID=YOUR_PROJECT_ID
```

您可以使用下列範例指令建立檔案。請依據狀況更換所有憑證：

```
cat > .env <<'EOF'
PBL_BACKEND_PORT=3000
LOOKERSDK_API_VERSION=4.0
LOOKERSDK_BASE_URL=YOUR_LOOKER_INSTANCE_URI_WITH_TRAILING_SLASH
LOOKERSDK_CLIENT_ID=YOUR_LOOKER_CLIENT_ID
LOOKERSDK_CLIENT_SECRET=YOUR_LOOKER_CLIENT_SECRET

LOOKERSDK_EMBED_HOST=YOUR_LOOKER_INSTANCE_URI
LOOKERSDK_EMBED_SECRET=YOUR_LOOKER_EMBED_SECRET

CLOUD_AGENT_ID=looker_embed_reference_data_agent
CLOUD_PROJECT_ID=YOUR_PROJECT_ID
EOF
```

8. 執行後端開發伺服器：

```
yarn run dev
```

您現在已啟動前端開發伺服器，為網頁應用程式提供 JavaScript。您也需要執行後端開發伺服器，處理單一登入 (SSO) 嵌入網址要求，並將要求 Proxy 至對話式數據分析端點。

4. 試用範例

現在，請試用在本機環境中執行的網頁應用程式。

發起對話

1. 在您選擇的瀏覽器中開啟 <https://localhost:3001> 位址。
2. 前往左側主要導覽中的「Embedded Dashboard with Chat」頁面。
3. 右側的即時通訊元件載入後，請輸入「你好，你是誰？」。
4. 請注意回覆內容。

應用程式會自動建立對話式數據分析 **conversation** 物件，追蹤即時通訊記錄並載入對話介面。您在聊天介面中提問時，前端會將使用者訊息傳送至本機 Node 後端伺服器。然後將要求和回應從對話式數據分析 **chat** 端點代理。

篩選嵌入式資訊主頁

現在請熟悉內嵌資訊主頁，並與其互動。

5. 捲動瀏覽嵌入的資訊主頁。請注意，這份報表涵蓋多個維度和指標的人口普查資料。
6. 您可以在資訊主頁左上方依「州」和「郡」篩選資訊主頁。將資訊主頁篩選器設為「Texas」。然後選取新顯示的藍色醒目顯示圓形箭頭按鈕，重新執行資訊主頁。
7. 請注意，所有圖表資料都已根據「德州」篩選。

根據資訊主頁內容提問

篩選資訊主頁後，接著繼續調查資訊主頁中的資料。

8. 在對話中輸入「告訴我租金最低的 5 個縣市」。
9. 請注意，您提供給對話式數據分析的提示詞現在包含額外字詞：「Filter on dimension 'state.state_name' being Texas.」（篩選維度「state.state_name」為 Texas 的資料）。
10. 現在請注意，回應的查詢和資料會依「德州」這個州別篩選。
11. 此外，在系統傳回資料結果後，內嵌資訊主頁會重新執行，並將「County」篩選器設為結果資料中定義的 5 個縣市。

現在您可以繼續調查人口普查資料，系統已為您篩選出 5 個郡。

恭喜！您已設定並試用嵌入式資訊主頁的簡單範例，該資訊主頁已整合對話式數據分析的對話。

5. 建構即時通訊元件

首先，請探索 `Chat` 元件，瞭解幕後運作方式。我們假設您瞭解如何使用 [Looker Embed SDK](#) 嵌入 Looker 資訊主頁。

使用 Looker 元件程式庫

1. 在慣用的 IDE 或終端機中，開啟 `Frontend/src/components/EmbedChat/components/chat.js` 的 `Chat` 元件檔案。
2. `Chat` 元件是以 [Looker 元件程式庫](#) 套件中的標準 [Looker UI React 元件](#) 建構而成。

[Looker 元件程式庫](#) 提供預先建構的 React 元件，可與嵌入式 Looker 資訊主頁中的樣式和元件保持一致，讓您輕鬆完美整合 Looker 內容和應用程式。

傳送使用者訊息

聊天介面必須將使用者的提示傳送至對話式數據分析。

3. 在 `Chat` 元件檔案的底部附近，`Chat` 元件包含 `ChatInput` 子元件，可為使用者提示提供輸入欄位。
4. 提交時，`showAndSendUserPrompt` 方法會將使用者的提示傳送至對話式數據分析 (透過 Node 後端代理)。

串流及顯示系統訊息

使用者傳送訊息給對話式數據分析後，我們需要顯示回覆。

5. 在 `Chat` 元件檔案底部附近，`Chat` 元件包含 `MessageList` 子元件，其中含有根據訊息類型顯示對話式數據分析訊息的邏輯。
6. `streamAndParseResponse` 方法會處理回應，不斷嘗試從串流 JSON 回應中剖析有效的系統訊息。成功剖析有效的系統訊息後，系統就會在 `MessageList` 中顯示該訊息。

為提供更即時的使用者體驗，我們強烈建議您在收到對話式數據分析的系統訊息時，立即顯示這些訊息。

對話式分析功能會根據使用者提示，傳送多則系統訊息，內容涵蓋查詢的資料來源、查詢、結果、視覺化內容等。等待所有系統訊息送達，可能會導致使用者認為發生效能或應用程式問題。

您現在已瞭解如何使用 Looker 元件程式庫建構 `Chat` 元件，傳送使用者訊息並串流回覆。

6. 將資訊主頁篩選器傳送至對話式數據分析

現在來瞭解如何在使用者提示中加入資訊主頁篩選器，讓對話式數據分析功能根據資訊主頁的內容 (篩選器) 篩選查詢。

監聽資訊主頁的篩選器變更事件

1. 在 `Frontend/src/components/EmbedChat/EmbedChat.js`，開啟 `EmbedChat` 元件檔案，代表應用程式的頁面，其中包含與先前探索的 `Chat` 元件整合的內嵌資訊主頁。
2. `EmbedChat` 元件會使用 `Embed SDK` 的 `.on(...)` 方法，監聽內嵌資訊主頁的 `"dashboard:filters:changed"` 事件。接著，元件會將目前的篩選器儲存在篩選器狀態中。

將篩選器狀態傳送至對話式數據分析

3. `EmbedChat` 元件會將篩選器傳遞至 `Chat` 元件，後者會將每個篩選器轉換為 `showAndSendUserPrompt` 方法中的 `"Filter on dimension '...' being ..."` 等字串，並附加至使用者的提示。

資訊主頁篩選器的名稱或鍵不一定與實際維度的名稱相同。您必須將篩選器的鍵對應至實際維度的名稱，對話式數據分析才能瞭解要篩選的維度。

以這個範例來說，在 `Chat` 元件中，`FIELD_FILTER_MAP` 物件會將維度名稱 `"state.state_name"` 對應至篩選器的鍵 `"State 📍"`。接著，`showAndSendUserPrompt` 方法會使用對應表，判斷要篩選的維度名稱。

7. 透過即時通訊控制資訊主頁篩選器

最後，我們來看看如何啟用 `Chat` 元件，以控制嵌入式資訊主頁的篩選器。

決定要設定哪些篩選器

1. `Chat` 元件的 `streamAndParseResponse` 方法一律會檢查系統訊息，確認是否含有對話式數據分析的資料結果。
2. 每當 `streamAndParseResponse` 方法剖析出含有資料結果的系統訊息時，就會檢查資料中的維度是否與篩選器中的維度相符。
3. 如果是，`streamAndParseResponse` 方法會將資料欄轉換為資訊主頁篩選器。這次方法會再次使用 `FIELD_FILTER_MAP`，但會反向轉換，也就是從維度名稱轉換為篩選條件的鍵。篩選器的值是資料欄中的值。

將篩選器變更事件傳送至內嵌資訊主頁

4. 使用所需資訊主頁篩選器時，`streamAndParseResponse` 方法會使用所需篩選器呼叫 `Chat` 元件的 `setFilters` 方法。
5. 這會呼叫 `EmbedChat` 元件的 `setDashboardFilters` 方法，並使用 `Embed SDK` 的 `send` 方法，將兩個事件 (含篩選器的 `"dashboard:filters:update"` 和 `"dashboard:run"`) 立即接連傳送至內嵌資訊主頁。
6. `"dashboard:filters:update"` 事件會變更內嵌資訊主頁的篩選器，而 `"dashboard:run"` 事件則會使用新篩選器重新執行資訊主頁的查詢。

你必須在 `"dashboard:filters:update"` 事件後傳送 `"dashboard:run"`。 `"dashboard:filters:update"` 事件不會自動重新執行資訊主頁的查詢。

8. 結論與重點

您已設定嵌入式 Looker 資訊主頁的工作範例，並整合對話式數據分析 Chat。您已瞭解如何透過嵌入式 Looker 資訊主頁和對話式數據分析，啟用自然語言驅動的資料探索功能。

- 您已使用 Looker 元件程式庫建構聊天元件。
- 您已將內嵌 Looker 資訊主頁的環境傳遞至對話式數據分析，方便探索資料。
- 您已啟用對話式數據分析，以便在探索資料時，控制內嵌資訊主頁的篩選器，取得更豐富的背景資訊。

後續步驟

- 進一步瞭解對話式數據分析的[功能](#)。
 - 更新 [Looker Embed 參考範例應用程式](#)，以便與您自己的嵌入式 Looker 資訊主頁搭配使用。
 - 試試在本機執行的 [Looker 嵌入參考資料](#)範例應用程式中，使用其他嵌入功能！
-